SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

Code-Layout, Readability and Reusability

Date	10 July 2026
Team ID	XXXXXXX
Project Name	AI Debt Relief Platform
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability

This document evaluates the quality of the codebase for the AI Debt Relief Platform in terms of its structure, readability and reusability.

The project follows modular coding practices, meaningful naming conventions, proper comments and reusable components to improve maintainability and scalability.

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code.	Yes	PEP-8 formatting followed.
2	Proper File Structure	Files and folders are logically organized (e.g., routes, models, schemas, services, utils).	Yes	Modular project structure maintained.
3	Meaningful Naming	Variables, functions, classes and routes have clear and meaningful names.	Yes	Descriptive naming convention followed.
4	Function / Method Names	Functions and methods are named based on their purpose and actions.	Yes	Readable API and function names used.

SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
5	Code Comments	Inline and block comments explain complex logic, AI models and important workflows.	Yes	Important modules and business logic documented.
6	Modular Design	Reusable functions, services and components are separated into modules.	Yes	Routes, services, models, utils, and schemas are well separated.
7	No Redundant Code	Duplicate or unused code is removed.	Yes	Refactored and optimized for clean code.
8	Error Handling	Exceptions and edge cases are handled gracefully.	Yes	Try/Except blocks and custom error responses implemented.

Overall Code Quality

The codebase of the AI Debt Relief Platform follows best practices for layout, readability and reusability, ensuring high maintainability, scalability and ease of future enhancements.

Key Strengths

- Well-structured and modular codebase
- Consistent naming and documentation
- Reusable components and clean architecture
- Proper error handling and input validation

Note

Regular code reviews and adherence to PEP-8 guidelines are recommended to maintain code quality and improve collaboration.